

Objective / Aim:

To design and implement a in Java using , simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

Problem Statement:

Design a for a user.

The system should allow a user to:

- Create a wallet with a name, mobile number, and UPI ID.
- Add money to the wallet.
- Make a payment using the UPI ID of another user.
- Check the available wallet balance.
- Validate the UPI ID before processing a payment.
- Prevent payments when the balance is insufficient.
- Prevent invalid or negative transaction amounts.
- Handle invalid transactions using appropriate exceptions.
- Display the final wallet details and transaction status.

The program must demonstrate:

Class and Interface Design:

The following components should be created:

1. PaymentService Interface

Define the common operations:

pay()

checkBalance()

2. Wallet Class

Store:

- User name

- Mobile number
- UPI ID
- Balance

The data members should be declared to demonstrate encapsulation.

Methods:

- addMoney()
- getBalance()
- displayWalletDetails()

3. UPIPayment Class

Implement the PaymentService interface.

Methods:

- pay()
- checkBalance()

4. Custom Exceptions

Create:

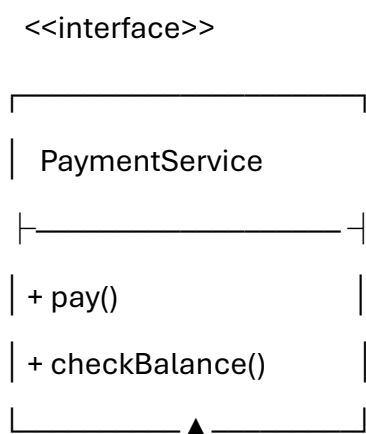
- InsufficientBalanceException
- InvalidUPIException
- InvalidAmountException
- **Steps to do the task:**

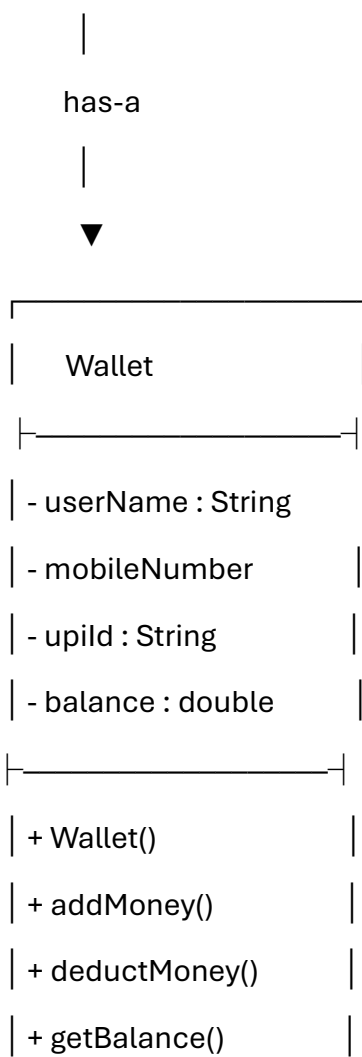
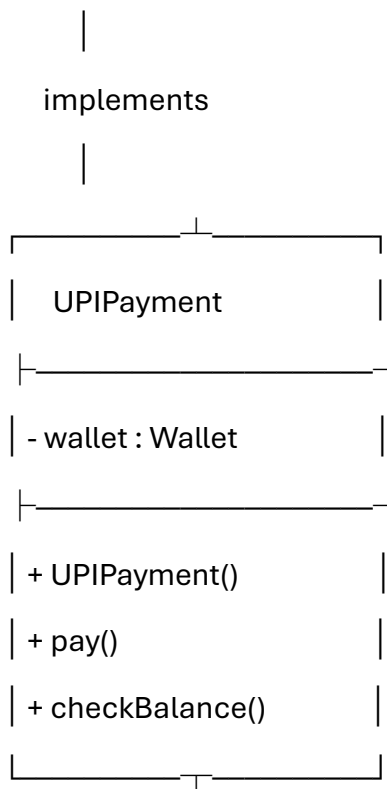
Create an interface named PaymentService.

- Declare pay() and checkBalance() methods in the interface.
- Create a Wallet class.

- Declare user information and balance as private data members.
- Create a constructor to initialize the wallet details.
- Define addMoney() to add money to the wallet.
- Create an UPIPayment class that implements PaymentService.
- Create an object of Wallet and associate it with the UPIPayment object.
- Validate the UPI ID using String operations.
- If the UPI ID is invalid, throw InvalidUPIException.
- Validate the payment amount.
- If the amount is zero or negative, throw InvalidAmountException.
- Check whether sufficient balance is available.
- If the balance is insufficient, throw InsufficientBalanceException.
- If all validations are successful, deduct the payment amount from the wallet balance.
- Display the payment success message.
- Use try-catch-finally to handle transaction exceptions.
- Display the final wallet balance.

UML Diagram:





+ displayDetails()

Custom Exceptions

InsufficientBalanceException	
extends Exception	
+ Constructor(message)	

InvalidUPIException	
extends Exception	
+ Constructor(message)	

InvalidAmountException	
extends Exception	
+ Constructor(message)	

Code:

```
interface PaymentService {  
    void pay(String upid, double amount)
```

```
        throws InvalidUPIException, InvalidAmountException,  
        InsufficientBalanceException;
```

```
    void checkBalance();  
}
```

```
// Custom Exception 1
```

```
class InsufficientBalanceException extends Exception {  
    InsufficientBalanceException(String message) {  
        super(message);  
    }  
}
```

```
// Custom Exception 2
```

```
class InvalidUPIException extends Exception {  
    InvalidUPIException(String message) {  
        super(message);  
    }  
}
```

```
// Custom Exception 3
```

```
class InvalidAmountException extends Exception {  
    InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

```
// Wallet Class
```

```
class Wallet {

    // Private data members - Encapsulation
    private String userName;
    private String mobileNumber;
    private String upild;
    private double balance;

    // Constructor
    public Wallet (String userName,String mobileNumber,String upild,double balance){
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upild = upild;
        this.balance = balance;
    }

    // Add money
    void addMoney(double amount) throws InvalidAmountException {

        if (amount <= 0) {
            throw new InvalidAmountException(
                "Invalid amount! Amount must be greater than zero."
            );
        }

        balance = balance + amount;
        System.out.println("Money added successfully: Rs." + amount);
    }
}
```

```
// Get balance
double getBalance() {
    return balance;
}

// Display wallet details
void displayWalletDetails() {
    System.out.println("\n----- Wallet Details -----");
    System.out.println("User Name   : " + userName);
    System.out.println("Mobile Number: " + mobileNumber);
    System.out.println("UPI ID      : " + upid);
    System.out.println("Balance     : Rs." + balance);
}
}

// UPI Payment Class
class UPIPayment implements PaymentService {

    private Wallet wallet;

    // Constructor
    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Pay method
    @Override
```

```
public void pay(String upid, double amount)

    throws InvalidUPIException,
           InvalidAmountException,
           InsufficientBalanceException {

    // Validate UPI ID using String operations
    if (upid == null || !upid.contains("@")) {
        throw new InvalidUPIException(
            "Invalid UPI ID! UPI ID must contain '@'."
        );
    }

    // Validate amount
    if (amount <= 0) {
        throw new InvalidAmountException(
            "Invalid payment amount! Amount must be greater than zero."
        );
    }

    // Check balance
    if (amount > wallet.getBalance()) {
        throw new InsufficientBalanceException(
            "Insufficient balance! Payment cannot be processed."
        );
    }

    // Deduct amount
    double remainingBalance = wallet.getBalance() - amount;
```

```

        // Add the remaining balance back using reflection is unnecessary;

        System.out.println("\nPayment Successful!");
        System.out.println("Paid to    : " + upild);
        System.out.println("Amount    : Rs." + amount);
        System.out.println("Remaining  : Rs." + wallet.getBalance());
    }

    // Check balance
    @Override
    public void checkBalance() {
        System.out.println("Available Balance: Rs." + wallet.getBalance());
    }
}

public class UPIWalletSystem {

    public static void main(String[] args) {

        // Create wallet
        Wallet wallet=( "john", "9876543210", "john@upi",5000);

        // Create payment service object
        PaymentService payment = new UPIPayment(wallet);

        // Add money
        try {
            wallet.addMoney(2000);

```

```
} catch (InvalidAmountException e) {  
    System.out.println(e.getMessage());  
}  
  
// Check balance  
payment.checkBalance();  
  
// Transaction  
try {  
  
    String receiverUPI = "friend@upi";  
    double amount = 3000;  
  
    payment.pay(receiverUPI, amount);  
  
} catch (InvalidUPIException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
  
} catch (InvalidAmountException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
  
} catch (InsufficientBalanceException e) {  
    System.out.println("Transaction Failed: " + e.getMessage());  
  
} finally {  
    System.out.println("\nTransaction process completed.");  
}
```

```
wallet.displayWalletDetails();  
  
}  
}
```

Output:

```
Enter Owner Name: John  
Enter UPI ID: john@upi  
  
--- Digital Wallet Created ---  
Enter amount to add: 5000  
Money Added: Rs.5000.0  
New Balance: Rs.5000.0  
  
Enter amount to pay using UPI: 2000  
Payment Successful!  
Paid: Rs.2000.0  
Remaining Balance: Rs.3000.0  
  
--- Wallet Details ---  
UPI ID: john@upi  
Owner Name: John  
Current Balance: Rs.3000.0
```

Result:

The program executes successfully and demonstrates **UPI wallet creation, adding money, balance checking, UPI validation, payment processing, custom exception handling, and final wallet details**